Tabular Deep Learning

Ivan Rubachev, Yandex Research

June 1, 2026

What the lecture is about

Annoucment: Deep Learning and Tabular Data

We'll talk about DL and tabular data. We'll go from California to $10 seed rounds.

- a broad outlook on the field
- talk about the field's evolution
- think about where it's going

Tehcnically speaking we'll also talk about

- DL model components specific to tabular data
- some IMO interesting speculations about why they are different (our group's most recent work)

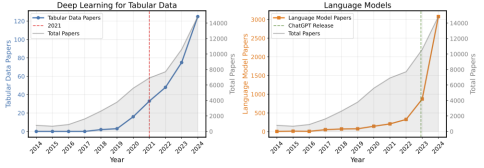
Later (seminar): Hands-on tabular DL — on a MacBook.

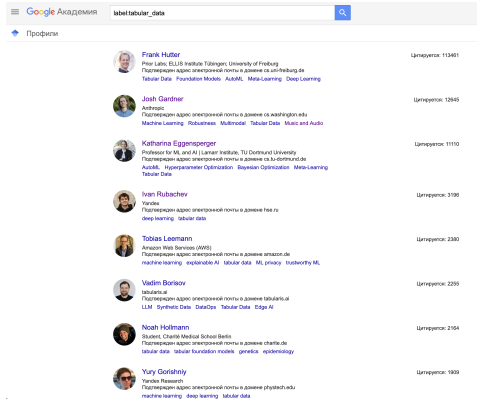
The idea:

- We implement some of the core models from scratch (or watch me ~~do~~ run it).
- The hardware constraint might help us appreciate the practical limits (or lack thereof).

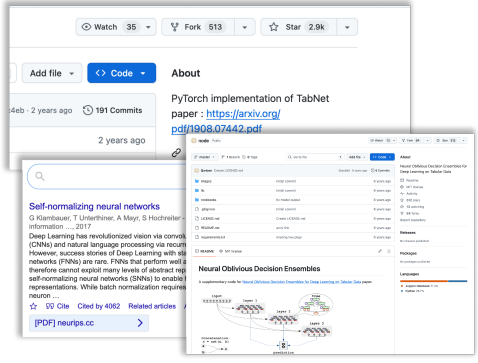
Before ~~it was~~ it started becoming cool

Attention to tabular data & deep learning intersection is increasing:





But a few years ago (~2021)...
I usually rant about this longer in lectures
This time I do this just to set up the "benchmarking is hard" (in *good* papers too) message plus it's fun to do some history, and to show *how we live*:



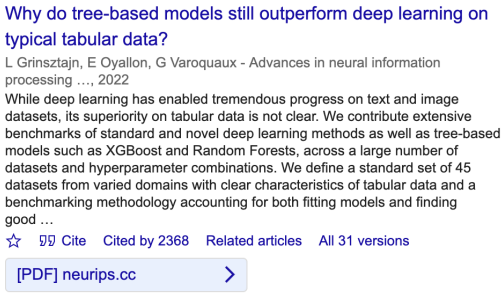
Once upon a time (a few years ago in ~2021)...

But...

Model	Average rank (std)
TabNet	7.5 (2.0)
SNN	6.4 (1.4)
AutoInt	5.7 (2.3)
GrowNet	5.7 (2.2)
MLP	4.8 (1.9)
DCN V2	4.7 (2.0)
NODE	3.9 (2.8)
ResNet	3.3 (1.8)
FT-Transformer	1.8 (1.2)

- Most methods fail to pass an MLP baseline
- Adding skip-connections
- Reusing the Transformer model (from totally unrelated domains)
- Works best (for whatever reason)

Why?



We'll recall this paper a bit later (when talking about analysis)

Now onto what works

GBDTs! fit() predict() is almost all you need

Bojan Tunguz
@tunguz

XGBoost is all you need.

But this lecture is about DL

Neural networks are different (as we've talked about a few moments prior)

There are basic things you should have in mind Especially when working with tabular NNs

---

Preprocessing

Much more important than for GBDTs

What you should know:

- Z-scoring (StandartScaler) or MinMaxScaler are often not enough
- There are more universally good methods but it's still super dataset specific

But there is also:

- `https://github.com/dholzmueller/pytabkit` for the RobustScale + Clip transform
- look at the `https://github.com/PriorLabs/TabPFN/tree/main/src/tabpfn/preprocessors` for inspiration
- you could also do quantile transform online and fast (if you have too much data): `https://arxiv.org/abs/1902.04023`
- it's probably not over (we'll talk a bit about this while discussing feature-embeddings) (e.g. Stretch Transformation)

If you want faster preprocessing for medium-large datasets look at `https://github.com/vaaven/fqt`

Dealing with missing features, or noisy features

Imputation for prediction: beware of diminishing returns

https://arxiv.org/abs/2407.19804

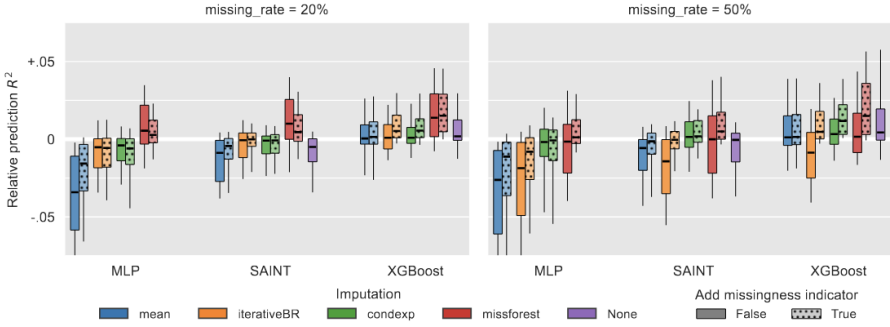


Figure 1: **Relative prediction performances across datasets for different imputations, predictors, and use of the missingness indicator.** Each boxplot represents 200 points (20 datasets with 10 repetitions per dataset). The performances shown are R^2 scores on the test set relative to the mean performance across all models for a given dataset and repetition. A value of 0.01 indicates that a given method outperforms the average performance on a given dataset by 0.01 on the R^2 score. Corresponding critical difference plots in figs. 6 and 7.

Noisy features `https://arxiv.org/abs/2311.05877`

- GBDT/Forset based feature selection and then use your model
- Proposed Deep Lasso (also kind-a works)
- Needs testing in the real world

Early stopping

- large enough validation sets (or cross-val based stopping if you deal with smaller data)
- think about the metric you early-stop on
- now it seems crucial
  - both ends: *sometimes* it may improve your methods significantly, *sometimes* it may be important to stop conservatively

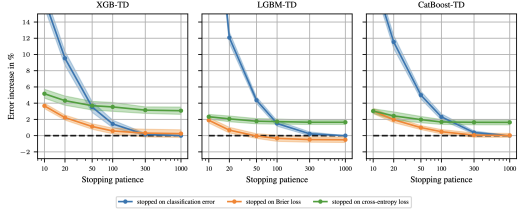


Figure B.1: **Effect of stopping patiences and metrics on the performance of GBDTs on $B^{\text{train}}_{\text{class}}$.** We run the XGB-TD, LGBM-TD, and CatBoost-TD with different early stopping patiences (`early_stopping_rounds`). We compare three different metrics used for stopping and best-epoch selection: classification error, Brier loss, and cross-entropy loss. The y-axis reports the relative increase in the benchmark score relative to stopping on classification error with patience 1000 (i.e., never stopping early). The shaded areas are approximate 95% confidence intervals, cf. Appendix C.6.

See also, a typical train, val and test curves on a tabular dataset:

Hyperparameter Tuning

This applies to GBDTs as well.

- BayesOpt is superior to random search (`https://proceedings.mlr.press/v133/turner21a.html`)
- With smaller models you *can* parallelize your tuning (modestly, due to perf reasons)
- You can use (and possibly will use in the future) tabular models as surrogates (see `https://arxiv.org/abs/2505.20685`)
- `https://puffer.ai` Protein and general direction is worth following (although it's in RL, it *feels* similar)
  - `https://x.com/jsuarez/` -

We'll take a look at a sane default during the seminar.

Tabular DL

Now we'll discuss some *common* building blocks important for tabular DL

Embeddings for numerical features

Introduced in `https://arxiv.org/abs/2203.05556`

In the FT-Transformer days: FT - feature tokenizer

TabM

Efficient ensembling.

First introduced in computer vision to make ensembling efficient

https://openreview.net/forum?id=Sklf1yrYDr

Published as a conference paper at ICLR 2020

BATCHENSEMBLE: AN ALTERNATIVE APPROACH TO EFFICIENT ENSEMBLE AND LIFELONG LEARNING

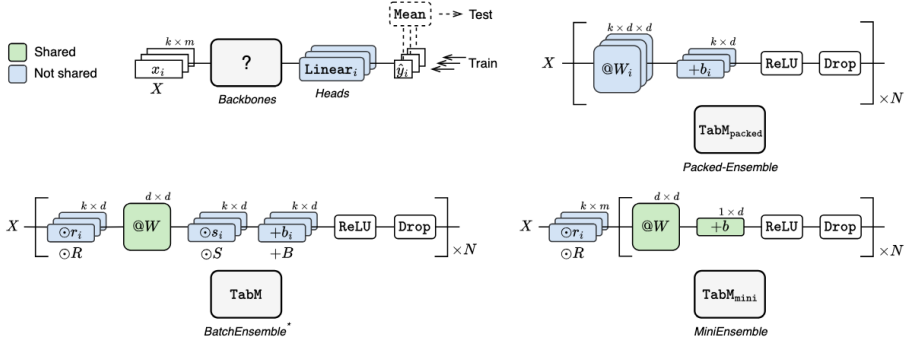
Yeming Wen[1,2,3*], **Dustin Tran**[3] **& Jimmy Ba**[1,2]
[1]University of Toronto, [2]Vector Institute, [3]Google Brain

ABSTRACT

Ensembles, where multiple neural networks are trained individually and their predictions are averaged, have been shown to be widely successful for improving both the accuracy and predictive uncertainty of single neural networks. However, an ensemble's cost for both training and testing increases linearly with the number of networks, which quickly becomes untenable.

In this paper, we propose BatchEnsemble[1], an ensemble method whose computational and memory costs are significantly lower than typical ensembles. BatchEnsemble achieves this by defining each weight matrix to be the Hadamard product of a shared weight among all ensemble members and a rank-one matrix per member. Unlike ensembles, BatchEnsemble is not only parallelizable across devices, where one device trains one member, but also parallelizable within a device, where multiple ensemble members are updated simultaneously for a given mini-batch. Across CIFAR-10, CIFAR-100, WMT14 EN-DE/EN-FR translation, and out-of-distribution tasks, BatchEnsemble yields competitive accuracy and uncertainties as typical ensembles; the speedup at test time is 3X and memory reduction is 3X at an ensemble of size 4. We also apply BatchEnsemble to lifelong learning, where on Split-CIFAR-100, BatchEnsemble yields comparable performance to progressive neural networks while having a much lower computational and memory costs. We further show that BatchEnsemble can easily scale up to

https://arxiv.org/abs/2410.24210



Retrieval

Once again, inspired by DL progress (RETRO for a good example)
`https://arxiv.org/abs/2112.04426`

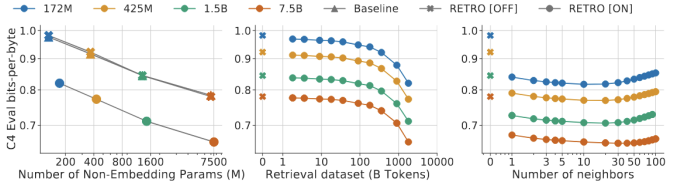


Figure 1 | **Scaling of RETRO.** The performance gain of our retrieval models remains constant with model scale (left), and is comparable to multiplying the parameteric model size by ~ 10×. The gain increases with the size of the retrieval database (middle) and the number of retrieved neighbours (right) on the C4 validation set, when using up to 40 neighbours. Past this, performance begins to degrade, perhaps due to the reduced quality. At evaluation RETRO can be used without retrieval data (RETRO[OFF]), bringing limited performance degradation compared to baseline transformers.

TabDL instantiations:

- TabR: https://arxiv.org/abs/2307.14338
- ModernNCA: https://arxiv.org/abs/2407.03257

Reaching for explanations

1. Back to why? `https://arxiv.org/abs/2207.08815`
   - Noisy features
   - Axis-aligned datasets
2. Uncertainty Lens `https://arxiv.org/abs/2509.04430`
   - Label noise (a.k.a. Data Uncertainty).
   - a CIFAR10 example
   - why TabM helps?
3. Telescoping: `https://arxiv.org/abs/2411.00247`

On AutoML

That may be worth looking into, if end users are less competent

- `https://github.com/sb-ai-lab/LightAutoML`
- `https://github.com/autogluon/autogluon`
- `https://catboost.ai` :)))
- `https://github.com/priorLabs/tabpfn`

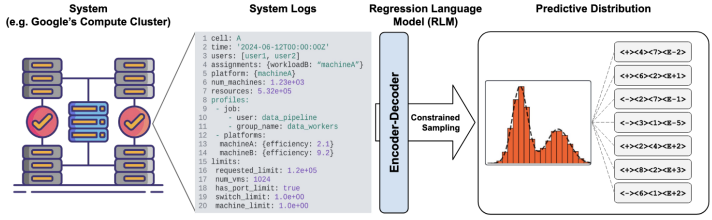
How do you define automl? what are the pros and cons

On foundation models for tabular data

Short forray and note on all sorts of LLMs + Tables + scaling lines of work for improving predictive ML accuracy.

- Automatic Feature Engineering (seems gimicky, but *may* be improved in the future)
- Finetuning Language Models to take on tabular data directly
- Wrangling and constructing Trees (e.g.)
- Using very small LMs as encoding of dirty text (see also `https://skrub-data.org/`)

Performance Prediction for Large Systems via Text-to-Text Regression
https://arxiv.org/abs/2506.21718



ICL-based foundation models

TabPFN and follow-up work

- TabPFNv2 (2.5)
- LimiX
- TabICL
- TabDPT
- OrionMSP
- MotherNet
- iLTM
- ... (many more yet to come)

But you can just read papers for this (if we have time during the seminar we'll talk about it).

What are ICL-based tabular foundation models (short recap)

A Prior-Fitted Network approximates Bayesian inference in a single forward pass.

> *Instead of fitting a model to your data, fit a model to all possible datasets from a prior.*

The Core Idea

Given a training set $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$ and a query point x_*:

Training Procedure

1. Define a prior $p(\theta)$ over data-generating processes
2. Sample synthetic datasets $\mathcal{D}^{(j)} \sim p(\mathcal{D}|\theta^{(j)})$
3. Train transformer to minimize:

$$\mathcal{L} = \mathbb{E}_{\theta,\mathcal{D},(x_*,y_*)} \left[-\log q_\phi(y_*|x_*, \mathcal{D}) \right] \tag{1}$$

where q_ϕ is the PFN with parameters ϕ.
https://www.nature.com/articles/s41586-024-08328-6

What is even generalized?
How scalable it is?
Nobody knows.
meta-learning + synthetic data feels like a nerd-snipe for years (for me personally).
So this is an idea and some questions before we end the lecture.

- Pretrained Transformers as Universal Computation Engines https://arxiv.org/abs/2103.05247
- Between Circuits and Chomsky: Pre-pretraining on Formal Languages Imparts Linguistic Biases https://arxiv.org/abs/2502.19249
- Universal pre-training by iterated random computation https://arxiv.org/abs/2506.20057
- Can You Learn to See Without Images? Procedural Warm-Up for Vision Transformers https://arxiv.org/abs/2511.13945

Hardware lottery `https://arxiv.org/abs/2009.06489`
The idea here is investment in software+hardware+research may drive the successs of methods. We may be seing something like this in deep learning approaches for tabular data right now (as we've tried to see during the seminar, software-wise iterating upon DL methods is much more aproachable, which may have downstream effects in research, that we've covered durign the lecture).